Task 1 – IaC Pipeline: AKS Cluster with ACR & Internal Ingress Controller

Task Description

The task was to create an automated pipeline that provisions an **Azure Kubernetes Service (AKS)** cluster using **Infrastructure as Code**. The pipeline needed to deploy:

- An **AKS cluster**
- An **ingress controller** (any type)
- An **Azure Container Registry (ACR)**

I had the option to use either **GitHub Actions** or **Azure DevOps Pipelines** for the implementation.

> **Bonus requirement**: Deploy the AKS cluster in **private network mode**, meaning the Kubernetes API server should **not be accessible from the public internet**.

Solution Overview

To implement the task, I chose to use **GitHub Actions** for automating the deployment process.

I decided to go beyond the basic requirements and include the **bonus task** of deploying the AKS cluster in **private network mode**. As a result, I created two separate GitHub Actions pipelines:

1. **Public AKS Pipeline** (`deploy-aks.yml`)
   Provisions an AKS cluster with public access to the Kubernetes API server. This pipeline includes:
   - AKS cluster creation
   - Azure Container Registry (ACR) setup
   - Installation of NGINX ingress controller
2. **Private AKS Pipeline** (`deploy-private-aks.yml`)
   Deploys an AKS cluster with a **private API endpoint**, meaning the Kubernetes API server is only accessible from within a specific virtual network. To interact with the private cluster from outside the VNet (e.g., during pipeline execution), I created a **jumpbox VM** that:
   - Is deployed **inside the same VNet and subnet** as the AKS cluster
   - Has a **public IP address** so that it can be accessed over SSH from GitHub-hosted runners

Both pipelines are triggered manually using `workflow_dispatch` and accept input parameters such as resource group name, cluster name, region, and monitoring preferences. This makes the workflows reusable and adaptable across different environments.

Compared to the public AKS pipeline, the private version introduces additional steps for setting up network isolation and a jumpbox VM to manage and access the cluster securely from inside the network. While the public pipeline is simpler and accessible over the internet, the private pipeline is designed for secure, internal-only environments where exposing the cluster externally is not allowed.

Prerequisites

Before implementing the pipeline, I completed the following setup steps to ensure I had the necessary resources and permissions in Azure:

1. Created a Free Azure Account

To begin, I signed up for a **free Azure account** at https://azure.microsoft.com/free. I then proceeded with the initial setup using the Azure Cloud Shell, which offers a convenient CLI interface pre-configured with all the necessary tools for Azure management.

2. Created a Service Principal

A **Service Principal** is a security identity used by applications, services, and automation tools to access Azure resources. It allows the pipeline to authenticate and perform actions on my behalf.

I created the service principal using the following command:

```
az ad sp create-for-rbac \
  --name "github-actions-deployer" \
  --role contributor \
  --scopes /subscriptions/$(az account show --query id -o tsv) \
  --sdk-auth
```

This command generates a JSON object that contains the credentials needed to authenticate:

```
{
  "clientId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "clientSecret": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
  "subscriptionId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "tenantId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  ...
}
```

I then used this output to create a repository secret named `AZURE_CREDENTIALS`. This secret is used by the GitHub Actions workflow to authenticate with Azure during pipeline execution.

I configured this secret in the repository under
Settings → Secrets and variables → Actions:

Pipeline: In-Depth Explanation

Step 1: `workflow_dispatch` with Inputs

```
on:
  workflow_dispatch:
    inputs:
      ...
```

Input Parameters Explained

The pipeline uses the `workflow_dispatch` trigger with several **input parameters** that make it flexible and reusable. Here's a detailed explanation of each parameter and its **purpose**:

`resourceGroup`

- **Description:** Name of the Azure Resource Group where all resources will be deployed.
- **Purpose:** Azure resources must be deployed inside a resource group. This parameter lets you choose or reuse a resource group to logically group and manage all created infrastructure.

`cluster`

- **Description:** The name of the AKS (Azure Kubernetes Service) cluster to create.

`acr`

- **Description:** The name of the Azure Container Registry (ACR).
- **Purpose:** ACR is a managed Docker registry service in Azure. It’s used to store container images and OCI artifacts like Helm charts. In Task 2, this ACR will be used to store the built application image and push Helm charts that will later be deployed to the AKS cluster.

`location`

- **Description:** Azure region where all resources will be deployed.
- **Default:** `Israel Central`
- **Options:** `eastus`, `Israel Central`
- **Purpose:** Specifies the geographic region where your infrastructure will run. It’s important to choose:

- A region close to your end users to reduce latency when accessing the application.
- The same region for all components (AKS, ACR, VM) to ensure faster internal communication between services, avoid region mismatch errors, and minimize cross-region data transfer costs.

`enableMonitoring`

- **Description:** Whether to enable monitoring with Log Analytics (`true` or `false`).
- **Default:** `true`
- **Purpose:** Enables Azure Monitor integration for the AKS cluster. This provides insights into cluster health, logs, resource usage, and performance.

`vnetName`

- **Description:** Name of the Virtual Network (VNet) where the AKS cluster and jumpbox VM will be deployed.
- **Default:** `aks-vnet`
- **Purpose:** Lets you control the network environment used by AKS and the VM. A VNet is a private, isolated network space within Azure. When deploying an AKS cluster in private mode, Azure requires that the cluster be placed inside a custom VNet to ensure the Kubernetes API server is not exposed to the internet.

`subnetName`

- **Description:** Name of the subnet within the VNet.
- **Default:** `aks-subnet`
- **Purpose:** Specifies the subnet where both the AKS cluster and the jumpbox will reside. Having a named subnet makes it easier to manage networking policies and assignments.A subnet is a smaller, segmented range within the VNet that Azure resources are deployed into. A custom subnet is required because:

The AKS nodes and the jumpbox VM must be placed in the same internal network to allow secure and direct communication.

Azure needs a specific subnet to assign private IP addresses to AKS control plane components and worker nodes.

Having a named subnet allows for fine-grained control over routing, firewall rules, network security groups (NSGs), and future network policies.

`subnetPrefix`

- **Description:** CIDR address range for the subnet (e.g., `10.240.0.0/16`)
- **Default:** `10.240.0.0/16` The 10.x.x.x range is part of the private IP address space (RFC 1918), so it’s not routable from the internet — perfect for internal communication.
- **Purpose:** Defines the address range from which internal IPs are assigned to cluster nodes and VMs. It's important for IP planning and VNet organization.

`nodeCount`

- **Description:** Number of VM nodes to create in the AKS cluster.
- **Default:** `1`
- **Purpose:** Controls the initial scale of the AKS cluster. A higher number provides more capacity for running workloads, while a lower number helps reduce cost for testing environments.

`nodeSize`

- **Description:** Azure VM size for each node in the AKS cluster (e.g., `Standard_B2s`)

- **Default:** `Standard_B2s` Standard_B2s is a low-cost, burstable VM with 2 vCPUs and 4 GB RAM, suitable for development, testing, and lightweight workloads. It can temporarily boost performance during high demand ("burstable"), making it both affordable and flexible.
- **Purpose:** Determines the compute and memory capacity of each worker node. You can select a size that fits your workload or budget requirements.

Step 2: `Log in to Azure`

```
- name: Log in to Azure
  uses: azure/login@v1
  with:
    creds: ${{ secrets.AZURE_CREDENTIALS }}
```

This uses the **Azure Login Action** to authenticate your GitHub workflow with Azure.

- It uses the credentials stored in the `AZURE_CREDENTIALS` secret.
- These are generated from a **Service Principal** with Contributor rights.

Purpose:
Every action that uses `az` CLI in the pipeline needs to be authenticated. This sets up that session securely.

Step 3: `Set environment variables`

```
echo "RG=${{ github.event.inputs.resourceGroup }}" >> $GITHUB_ENV
...
```

This step extracts the input values passed from the manual trigger and writes them into environment variables.

$GITHUB_ENV is a special file used in GitHub Actions to define environment variables that will be automatically available in all later steps of the same job.

Purpose:
Simplifies referencing these values in multiple scripts without repeating `${{ github.event.inputs.xxx }}` over and over.

Step 4: `Parse Azure Credentials`

```
echo "CLIENT_ID=$(echo '${{ secrets.AZURE_CREDENTIALS }}' | jq -r
.clientId)" >> $GITHUB_ENV
...
```

This extracts the **client ID**, **client secret**, and **tenant ID** from the JSON secret using `jq`.

Purpose:
These values are needed later — inside the **jumpbox VM**, where we must login to Azure again to connect to the private AKS cluster.

Step 5: `Create Resource Group`

```
EXISTS=$(az group exists --name "${{ env.RG }}")
...
az group create --name "${{ env.RG }}" --location "${{ env.LOCATION }}"
```

This checks if the resource group already exists. If not, it creates one in the selected region.

Purpose:
Resource groups are the **top-level container** for managing Azure resources. This ensures the rest of your infrastructure has a place to live.

Step 6: `Create Virtual Network and Subnet`

```
az network vnet create \
  --resource-group $RG \
  --name $VNET_NAME \
  --address-prefixes $ADDRESS_PREFIX \
  --subnet-name $SUBNET_NAME \
  --subnet-prefix $SUBNET_PREFIX
```

This creates a **virtual network** and a **subnet** inside it.

- `--address-prefixes` = entire VNet range (e.g. `10.0.0.0/8`)
- `--subnet-prefix` = smaller segment for AKS and VM (e.g. `10.240.0.0/16`)

Then it retrieves the **subnet ID** which is required to place the AKS cluster inside that private network.

Purpose:
A private AKS cluster must be placed inside a **custom subnet**. The default AKS subnet is public and won't work for private clusters.

Step 7: `Create ACR`

```
az acr create \
  --name $ACR_NAME \
  --resource-group $RG \
  --sku Basic \
  --location "$LOCATION"
```

This creates an **Azure Container Registry**.

- `--sku Basic`: Cheapest tier, good for development and small deployments.
- `--location`: Matching the AKS region for better latency and integration.

Purpose:
ACR is where you'll store the application's Docker images and Helm charts for deployment to AKS (in Task 2). It integrates directly with AKS, so your cluster can securely pull images without needing external registries like Docker Hub — making the setup faster, more secure, and fully within Azure.

Step 8: `Create Private AKS Cluster`

```
az aks create \
  --name $CLUSTER_NAME \
  --resource-group $RG \
  --location "$LOCATION" \
  --node-count $NODE_COUNT \
  --node-vm-size $NODE_SIZE \
  --network-plugin azure \
  --vnet-subnet-id $SUBNET_ID \
  --enable-private-cluster \
  --enable-managed-identity \
  --attach-acr $ACR_NAME \
  $MONITORING_FLAG \
  --generate-ssh-keys
```

This is the core step — creating the AKS cluster with all required flags:

- `--enable-private-cluster`: Ensures **Kubernetes API server is not public**
- `--vnet-subnet-id`: Places cluster into our private subnet
- `--attach-acr`: Grants access to pull from ACR securely
- `--enable-managed-identity`: Enables a managed identity for the AKS cluster, allowing it to securely access other Azure resources (like ACR) without needing to store credentials. Azure automatically handles the identity lifecycle, making it more secure and easier to manage.
- `--monitoring-flag`: Optional Log Analytics integration

Purpose:
This produces a **secure**, **isolated**, and **ready-to-use** AKS cluster, with all connections routed internally.

Step 9: `Create Jumpbox VM`

```
az vm create \
  --resource-group $RG \
  --name aks-jumpbox \
  --image Ubuntu2204 \
  --size Standard_B2s \
  --admin-username azureuser \
```

```
  --generate-ssh-keys \
  --vnet-name $VNET_NAME \
  --subnet $SUBNET_NAME \
  --public-ip-address aks-jumpbox-ip
```

Creates a simple **Ubuntu VM** in the **same subnet** as the AKS cluster.

- `--public-ip-address`: Makes it reachable from the GitHub runner
- `--generate-ssh-keys`: Keys stored locally to allow login in the next step

Purpose:
You cannot access a private AKS cluster directly. The jumpbox is your **bridge** for safe, SSH-based access.

Step 10: `SSH and Configure Jumpbox`

```
ssh -i ~/.ssh/id_rsa azureuser@$JUMPBOX_PUBLIC_IP ...
```

Inside the SSH session:

- Install tools: `curl`, `unzip`, `docker`, `helm`, `kubectl`, `az`
- Log into Azure using the same service principal
- Use `az aks get-credentials` to connect `kubectl` to AKS
- Install **NGINX Ingress** with an internal load balancer using Helm An internal ingress controller lets you securely route traffic to your applications inside the AKS cluster, but only from within the private Azure network (like the jumpbox or other trusted services). It's perfect when your workloads are private or when you're setting up a secure internal platform.

```
helm upgrade --install internal-ingress bitnami/nginx-ingress-controller \
  --namespace ingress-nginx \
  --create-namespace \
  --set service.type=LoadBalancer \
  --set-string service.annotations."service\.beta\.kubernetes\.io/azure-
load-balancer-internal"="true" \
  --set rbac.create=true \
  --set controller.publishService.enabled=true
```

Here's a breakdown of what each part of the command does:

upgrade --install internal-ingress: Installs the release named internal-ingress.

bitnami/nginx-ingress-controller: Specifies the Helm chart source — in this case, the NGINX ingress controller provided by Bitnami.

--namespace ingress-nginx: Deploys the ingress controller into the ingress-nginx namespace.

--create-namespace: Creates the ingress-nginx namespace automatically if it doesn't already exist.

--set service.type=LoadBalancer: Tells Kubernetes to expose the ingress controller service via a LoadBalancer, which in Azure will provision a load balancer.

--set-string service.annotations."service.beta.kubernetes.io/azure-load-balancer-internal"="true": Adds an annotation to the service that instructs Azure to provision an internal load balancer rather than a public one. This ensures traffic is only allowed inside the virtual network.

--set rbac.create=true: Ensures that the necessary Role-Based Access Control (RBAC) resources are created for the ingress controller to function properly.

--set controller.publishService.enabled=true: Enables publishing of the ingress controller service's IP address, which is necessary for routing traffic through the internal load balancer.

Purpose:
This step sets up a **fully functional private ingress controller**, needed to expose your apps within the cluster via internal IPs.
